DOCUMENTATION TECHNIQUE

2026

1. Introduction générale

Le projet **CryptoVault** est une application de coffre-fort numérique sécurisé développée dans le but de proposer une solution fiable de stockage, de gestion et de partage de fichiers sensibles. Dans un contexte où les données personnelles et professionnelles sont de plus en plus dématérialisées, la sécurisation des documents numériques est devenue un enjeu majeur, aussi bien pour les particuliers que pour les entreprises.

En effet, la multiplication des cyberattaques, des vols de données et des obligations réglementaires telles que le RGPD impose aujourd'hui des exigences élevées en matière de confidentialité, d'intégrité et de contrôle des accès. CryptoVault s'inscrit directement dans cette problématique en mettant à disposition un espace de stockage chiffré, accessible uniquement par des utilisateurs authentifiés, et intégrant des fonctionnalités avancées telles que le versionnage automatique, les quotas utilisateurs ainsi que le partage public contrôlé via des liens temporaires.

Cette documentation a pour objectif de fournir une vision complète du projet, aussi bien sur ses aspects fonctionnels que sur ses choix techniques. Elle est destinée à permettre à un développeur tiers de comprendre l'architecture du système, de l'installer, de le maintenir et de le faire évoluer.

2. Objectifs et fonctionnalités principales

CryptoVault a été conçu comme une solution complète articulée autour de plusieurs fonctionnalités essentielles. Tout d'abord, l'application permet à chaque utilisateur de disposer d'un espace personnel sécurisé, organisé sous forme de dossiers hiérarchiques. L'utilisateur peut y déposer des fichiers qui sont automatiquement chiffrés avant stockage, garantissant ainsi qu'aucune donnée ne soit conservée en clair sur le serveur.

Un second objectif majeur du projet est la mise en place d'un système de versionnage. Lorsqu'un fichier est remplacé, une nouvelle version est créée, permettant de conserver l'historique complet des modifications. Cette approche garantit non seulement la traçabilité, mais aussi la possibilité de restaurer des versions précédentes si nécessaire.

CryptoVault propose également un mécanisme de partage sécurisé. Un utilisateur peut générer un lien public associé à un fichier ou un dossier, avec des contraintes précises telles qu'une date d'expiration ou un nombre maximum de téléchargements. Ce lien donne accès à une page web publique minimaliste permettant au destinataire de récupérer le document, sans authentification directe.

Enfin, le projet intègre une gestion de quotas afin de limiter l'espace de stockage utilisé par chaque compte, ainsi qu'un système de journalisation permettant de tracer toutes les actions sensibles comme les téléchargements ou les accès publics.

3. Technologies et outils utilisés

Le projet CryptoVault repose sur une architecture full-stack moderne, structurée autour de plusieurs technologies adaptées aux exigences de sécurité.

Le backend est développé en ***PHP 8.2*** à l'aide du framework ***Slim 4,*** qui permet de construire une API REST légère, modulaire et facilement maintenable. La persistance des données est assurée par une base relationnelle ***MySQL/MariaDB,*** accédée via *Medoo*, un micro-ORM simplifiant les interactions tout en conservant un contrôle précis sur les requêtes.

La sécurité repose sur plusieurs briques : l'authentification est gérée par des tokens JWT, les mots de passe sont hachés avec ***Bcrypt*** via *password_hash()*, et les fichiers sont chiffrés grâce à AES-256-GCM couplé à un mécanisme d'enveloppe de clés basé sur une ***KEK côté serveur*** (Key Encryption Key)

Deux clients ont été développés : ***un client lourd JavaFX*** destiné aux utilisateurs authentifiés, ainsi qu'***une interface web publique*** légère construite en HTML/CSS (Bootstrap), permettant uniquement le téléchargement via token.

Le projet s'appuie également sur des outils professionnels : ***OpenAPI*** pour documenter le contrat de l'API, ***Docker*** pour faciliter le déploiement, ***Postman/Newman*** pour les tests E2E, ainsi que ***GitHub Projects*** pour l'organisation agile du développement.

4. Architecture globale du système

CryptoVault suit une architecture client-serveur classique, articulée autour d'un backend central jouant le rôle de source unique de vérité. Toute la logique métier, notamment la sécurité, le chiffrement, la validation des accès ou la gestion des quotas, est implémentée côté serveur.

Le système se compose de ***trois blocs principaux***. *Le premier est l'API REST sécurisée*, qui expose les endpoints nécessaires aux opérations principales : authentification, gestion des dossiers, upload et téléchargement de fichiers, versionnage et partage.

Le *second bloc* est la *base de données relationnelle*, qui stocke toutes les métadonnées du coffre-fort : utilisateurs, fichiers, versions, liens de partage et journaux d'audit.

Enfin, *le troisième bloc* correspond aux interfaces clientes : *le client JavaFX* complet pour la gestion du coffre, et l'interface web publique pour les téléchargements externes.
Cette architecture garantit une séparation claire des responsabilités, tout en assurant une extensibilité future.

5. Backend REST API : conception et endpoints

Le backend constitue le cœur du projet. Il a été conçu comme une API REST stateless, ce qui signifie que le serveur ne conserve aucune session : chaque requête est authentifiée via JWT.

L'API expose plusieurs familles d'endpoints. Les routes d'authentification permettent de créer un compte et de récupérer un token de connexion. Une fois connecté, l'utilisateur peut interagir avec les dossiers via des endpoints CRUD, puis déposer des fichiers grâce à un endpoint multipart dédié.

Lors d'un upload, le backend chiffre automatiquement le fichier avant stockage. Une version initiale est créée dans la table *file_versions*. Si un fichier est remplacé, l'endpoint *POST /files/{id}/versions* permet de générer une nouvelle version, en conservant l'historique complet.

Le backend fournit également un système de partage via *POST /shares,* qui génère un token sécurisé associé à des contraintes d'usage. Les routes publiques */s/{token}* permettent au destinataire d'accéder à la ressource.

Enfin, des endpoints complémentaires permettent de consulter le quota utilisateur ou d'accéder aux logs d'activité.

6. Sécurité et chiffrement des fichiers

La sécurité constitue la priorité centrale de ***CryptoVault***. Les fichiers stockés ne doivent jamais être accessibles en clair, même en cas de compromission du serveur ou d'accès non autorisé au support de stockage.

Pour répondre à cet objectif, *CryptoVault* utilise l'algorithme ***AES-256-GCM,*** un standard moderne de chiffrement authentifié. Ce mode garantit à la fois la confidentialité des données et leur intégrité grâce à la génération d'un tag d'authentification. Chaque fichier est chiffré avec une clé de contenu unique, générée aléatoirement lors de l'upload, ce qui empêche toute réutilisation de clé entre différents fichiers ou versions

Afin de protéger cette clé de contenu, CryptoVault met en œuvre un mécanisme d'***enveloppe cryptographique (key wrapping)***. La clé AES utilisée pour chiffrer le fichier est encapsulée à l'aide d'une ***clé maîtresse côté serveur (KEK – Key Encryption Key)***, elle-même utilisée uniquement pour chiffrer et déchiffrer les clés de fichiers. Cette enveloppe est réalisée via ***AES-256-GCM*** et stockée en base de données sous la forme d'un champ *key_envelope.*

Lors d'un téléchargement, le serveur utilise la KEK pour décapsuler la clé AES correspondante, puis procéder au déchiffrement du fichier. Sans cette clé maîtresse, il est impossible de récupérer la clé de contenu et donc d'accéder aux données en clair.

Ce modèle, basé sur une séparation entre clé de contenu et clé maîtresse, est couramment utilisé dans les systèmes de coffre-fort numérique car il combine performance, sécurité et maintenabilité cryptographique.

7. Client JavaFX : architecture et fonctionnalités avancées

Le client lourd JavaFX constitue l'interface principale de gestion du coffre-fort. Il repose sur une ***architecture MVC*** stricte : les vues sont définies en *FXML*, les contrôleurs gèrent les interactions, et un ApiClient centralise tous les appels REST.

L'application démarre sur un écran de connexion. Une fois authentifié, l'utilisateur accède à une interface principale structurée autour d'un explorateur de dossiers (*TreeView*) et d'une liste de fichiers (*TableView*).
Le client propose des fonctionnalités avancées telles que ***l'upload*** asynchrone avec barre de progression, le ***remplacement*** d'un fichier générant automatiquement une nouvelle version, ainsi qu'un module complet de ***gestion des partages***.

Une attention particulière est portée à l'expérience utilisateur : gestion des erreurs HTTP, affichage du quota utilisé, messages explicites en cas de dépassement ou d'accès refusé.

8. Interface web publique : téléchargement sécurisé

CryptoVault inclut une interface web publique minimaliste, destinée uniquement aux destinataires externes d'un lien de partage.

Lorsqu'un utilisateur génère un partage, un token est produit. Ce token permet d'accéder à une page de téléchargement qui affiche les informations essentielles : nom du fichier, taille, expiration éventuelle, nombre de version et bouton de téléchargement.

Cette page gère plusieurs états : lien valide, expiré, révoqué ou usages épuisés. Aucune authentification n'est nécessaire, ce qui rend l'expérience simple pour un destinataire externe tout en conservant un contrôle strict.

La sécurité web est renforcée par plusieurs mesures, notamment l'interdiction d'indexation par les moteurs de recherche, la neutralité des messages d'erreur afin de ne pas révéler d'informations sensibles, ainsi qu'une conception volontairement minimaliste limitant la surface d'attaque. Dans un contexte de déploiement en production, l'accès à cette interface publique doit être servi exclusivement via HTTPS afin de garantir la confidentialité des échanges et d'éviter toute interception du token de partage.

9. Justification technique des choix

Les choix techniques de CryptoVault ont été guidés par la recherche d'un équilibre entre robustesse, maintenabilité et conformité aux standards modernes.
Slim a été retenu pour sa légèreté et son adéquation aux API REST. ***JWT*** permet une architecture scalable sans sessions. ***AES-GCM*** est un standard industriel recommandé pour le chiffrement authentifié. Le mécanisme d'enveloppe de clés via ***KEK*** renforce la protection des clés de contenu

sans complexité asymétrique inutile. JavaFX apporte une interface riche et ergonomique, tandis que le client web public reste volontairement minimal afin de réduire la surface d'attaque.

10. Installation et mise en place du projet

Afin de permettre à un nouveau développeur de reprendre ***CryptoVault*** rapidement, il est essentiel de comprendre la procédure d'installation et de configuration du projet. *CryptoVault* a été conçu pour pouvoir être déployé dans un environnement local ou serveur avec une configuration relativement standard.

Le ***backend*** nécessite tout d'abord un ***environnement PHP*** récent (version 8.2 minimum), ainsi qu'un ***serveur web*** compatible tel que ***Nginx ou Apache***. L'application repose sur ***Slim Framework***, ce qui implique l'utilisation de ***Composer*** pour installer les dépendances.

Une fois le projet cloné depuis le dépôt Git, l'installation se fait en exécutant la commande :

- *composer install*

Le projet utilise un fichier de configuration ***.env*** permettant de centraliser les variables sensibles telles que :

- les identifiants de connexion à la base de données,
- la clé secrète utilisée pour signer les tokens JWT,
- la clé maîtresse serveur (KEK) utilisée pour encapsuler les clés de chiffrement des fichiers,

Cette approche permet de séparer clairement la configuration de l'application de son code source, ce qui est une bonne pratique indispensable en production.

La base de données doit ensuite être initialisée à l'aide des scripts de migration ou du fichier SQL fourni. Une fois les tables créées, le serveur peut être lancé, et un endpoint de test tel que « / » permet de vérifier que l'API fonctionne correctement.

CryptoVault peut également être exécuté dans un environnement containerisé via ***Docker Compose***, ce qui facilite la mise en place rapide d'un environnement reproductible.
Le lancement se fait simplement avec :

- *docker-compose up --build*

permet de lancer automatiquement :

- l'API PHP exécutée sous Apache,
- la base de données MySQL,
- ainsi qu'un service phpMyAdmin destiné au développement.

L'API devient alors accessible via :

- http://localhost:9081

Cette configuration Docker est principalement orientée démonstration et développement. Pour un déploiement en production, il serait recommandé d'ajouter une terminaison HTTPS via un reverse proxy (par exemple Nginx) et de renforcer davantage la gestion des secrets et des permissions.

11. Base de données et modèle relationnel

La base de données relationnelle joue un rôle central dans ***CryptoVault***, car elle stocke toutes les métadonnées nécessaires au fonctionnement du coffre-fort. Les fichiers eux-mêmes ne sont jamais enregistrés en clair dans la base, mais uniquement sur le disque sous forme chiffrée.

Le modèle repose sur plusieurs tables principales :

La table users contient les informations relatives aux comptes :

- email unique,
- hash du mot de passe (Bcrypt),
- quota total et quota utilisé,
- rôle administrateur éventuel.

La table folders permet de représenter une arborescence hiérarchique. Chaque dossier appartient à un utilisateur et peut contenir un dossier parent, ce qui permet de créer une structure similaire à un explorateur classique.

Les fichiers sont décrits dans ***la table files,*** qui contient les métadonnées générales :

- nom original,
- type MIME,
- dossier associé,
- date de création.

Cependant, *CryptoVault* se distingue par son système de versionnage : les fichiers ne sont jamais écrasés. Chaque modification génère une entrée dans ***la table file_versions.***
Cette table contient notamment :

- le numéro de version,
- le nom du fichier chiffré stocké sur disque,
- l'IV et le tag GCM,
- l'enveloppe ***AES-GCM via KEK*** de la clé de contenu (*key_envelope*) ,
- le checksum SHA-256.

Cette conception garantit l'immuabilité des versions et permet de conserver un historique complet.

Le partage est géré par ***la table shares,*** qui associe un token sécurisé à une ressource (fichier ou dossier). Elle stocke également les contraintes :

- expiration,
- nombre maximal d'usages,
- révocation.

Enfin, toutes les actions sensibles sont enregistrées dans ***downloads_log***, qui permet une traçabilité complète en stockant :

- date,
- adresse IP,

- user-agent,
- succès ou échec.

Ainsi, le modèle relationnel assure à la fois cohérence, sécurité et auditabilité.

12. Scénarios d'utilisation et workflow applicatif

Pour mieux comprendre le fonctionnement global de ***CryptoVault***, il est utile de présenter le scénario standard d'utilisation, qui correspond également au fil rouge de démonstration du projet.

Un utilisateur commence par ***créer un compte*** via l'endpoint d'inscription, puis ***se connecte*** afin d'obtenir un token JWT. Ce token est ensuite utilisé pour toutes les requêtes authentifiées.

Une fois connecté, l'utilisateur peut ***créer des dossiers*** afin d'organiser son coffre. Il peut ensuite ***uploader un fichier***. À ce moment, le backend chiffre automatiquement le contenu, génère une clé AES unique, encapsule cette clé via la ***KEK (key wrapping en AES-256-GCM)***. Le fichier est alors stocké sur le serveur sous forme chiffrée, et la ***version 1 est enregistrée*** dans la base.

Si l'utilisateur souhaite ***modifier un fichie***r existant, il utilise la ***fonctionnalité de remplacement.*** Cela déclenche la création d'une nouvelle version via l'endpoint dédié. L'ancienne version reste intacte, ce qui permet une traçabilité totale.

L'utilisateur peut ensuite ***générer un lien de partage public***. Ce lien contient un token sécurisé associé à des règles strictes (expiration ou nombre d'usages). Le ***destinataire*** peut ***ouvrir ce lien dans un navigateur*** et accéder à une page web publique minimaliste permettant le téléchargement.

Chaque téléchargement est journalisé, ce qui permet à l'utilisateur ou à un administrateur de suivre l'activité du coffre.

Enfin, ***le quota utilisateur*** est mis à jour à chaque opération afin d'éviter tout dépassement de stockage. Si le quota est atteint, l'upload est bloqué avec une réponse explicite.

Ce workflow illustre la cohérence globale du projet : chaque action utilisateur est sécurisée, tracée et conforme aux objectifs initiaux.

13. Perspectives d'évolution

Même si **CryptoVault** constitue déjà une solution fonctionnelle complète, plusieurs axes d'amélioration pourraient être envisagés dans une future version afin d'enrichir l'application et de renforcer encore davantage sa robustesse.

Sur ***le plan de la sécurité,*** il serait pertinent d'intégrer une ***authentification à double facteur (2FA)***, ainsi qu'un ***mécanisme de rotation des clés cryptographiques*** ou une intégration avec un service externe de gestion de clés (KMS). De plus, un monitoring avancé accompagné d'***alertes de sécurité*** permettrait une meilleure supervision en production.

Du point de vue des ***fonctionnalités applicatives,*** des améliorations pourraient inclure la mise en place d'un ***système de récupération de compte*** (« mot de passe oublié »), l'ajout d'un ***historique détaillé des actions*** utilisateur, ou encore la possibilité de ***restaurer automatiquement des versions*** précédentes.

Enfin, plusieurs ***optimisations ergonomiques*** pourraient renforcer l'expérience utilisateur, telles que le ***déplacement de dossiers et de fichiers,*** la possibilité de ***téléverser directement des fichiers à la racine du coffre***, ou encore l'application de règles plus strictes concernant la ***complexité des mots de passe.***

Ces perspectives démontrent que l'architecture actuelle a été pensée pour être extensible et qu'elle constitue une base solide pour de futures évolutions.

14. Conclusion

CryptoVault constitue une solution complète de coffre-fort numérique sécurisé, intégrant des mécanismes professionnels de chiffrement, de versionnage, de partage contrôlé et de traçabilité.
Le projet démontre une maîtrise de l'architecture REST, de la cryptographie appliquée, de la conception de bases de données, du développement client lourd JavaFX et du déploiement moderne.

Cette documentation fournit l'ensemble des éléments nécessaires pour permettre à un développeur tiers de comprendre le fonctionnement interne du système, d'en assurer la maintenance et d'envisager ses évolutions futures.